

سروناز حاجی محمدرضا
شماره دانشجویی: 40112012
پروژه سوم
درس نرم افزارهای ریاضی

ادرس پروژه در گیت

<https://github.com/sarvihmr/MathematicalSoftware-project3.git>

سوال اول

هدف این پروژه، پیاده‌سازی دو روش درونیایی لاگرانژ و اسپلاین مکعبی برای مجموعه نقاط $(1,1)$, $(2,3)$, $(3,5)$, $(4,8)$, $(5,5)$, $(6,2)$ و استخراج ضابطه دقیق هر روش است.

پیاده‌سازی روش لاگرانژ

برای پیاده‌سازی درونیایی لاگرانژ از تابع آماده lagrange موجود در کتابخانه scipy.interpolate استفاده شد. این تابع با دریافت بردار نقاط x و y ، چندجمله‌ای درونیایی را تولید می‌کند.

```
# Lagrange
poly = lagrange(x, y)

X = sp.Symbol('x')
expr = sum(sp.Float(c) * X**i for i, c in enumerate(poly.coef[::-1]))
expr = sp.expand(expr)

print("Lagrange Polynomial:\n")
print(sp.N(expr, 10))
print("\n" + "=" * 60 + "\n")
```

شکل 1

سپس برای نمایش ضابطه چندجمله‌ای، خروجی تابع به کمک کتابخانه SymPy به یک عبارت ریاضی تبدیل و به صورت نمادین چاپ شد.

مراحل انجام کار عبارت‌اند از:

1. تعریف نقاط داده .
2. فراخوانی تابع lagrange.
3. تبدیل خروجی به عبارت ریاضی .
4. چاپ ضابطه نهایی چندجمله‌ای .

Lagrange Polynomial:

$$0.175x^{**5} - 2.958333333x^{**4} + 18.375x^{**3} - 52.04166667x^{**2} + 68.45x - 31.0$$

=====

شکل 2

پیاده‌سازی روش اسپلاین

برای پیاده‌سازی اسپلاین از کلاس CubicSpline موجود در کتابخانه scipy.interpolate استفاده شد.

در این روش، برخلاف لاگرانژ، تنها یک چندجمله‌ای برای کل بازه ساخته نمی‌شود، بلکه برای هر بازه بین دو نقطه متوالی یک چندجمله‌ای درجه سه محاسبه می‌شود. بنابراین برای شش نقطه، پنج چندجمله‌ای مستقل به دست می‌آید.

پس از محاسبه اسپلاین، ضرایب هر بازه استخراج شده و با استفاده از کتابخانه SymPy، ضابطه هر چندجمله‌ای به صورت فرمول ریاضی چاپ گردید.

Spline interpolation:

1.0 <= x <= 2.0

$0.666666667x^3 - 4.0x^2 + 9.333333333x - 5.0$

2.0 <= x <= 3.0

$0.666666667x^3 - 4.0x^2 + 9.333333333x - 5.0$

3.0 <= x <= 4.0

$-2.333333333x^3 + 23.0x^2 - 71.66666667x + 76.0$

4.0 <= x <= 5.0

$1.666666667x^3 - 25.0x^2 + 120.3333333x - 180.0$

5.0 <= x <= 6.0

$1.666666667x^3 - 25.0x^2 + 120.3333333x - 180.0$

شکل 3

مراحل اجرای این بخش عبارت‌اند از:

1. تعریف نقاط داده .
2. ایجاد شیء CubicSpline.
3. استخراج ضرایب هر بازه .
4. تشکیل معادله هر اسپلاین .
5. بسط و چاپ فرمول مربوط به هر بازه.

```

# Cubic Spline
cs = CubicSpline(x, y)

X = sp.Symbol('x')

print("Spline interpolation:\n")

for i in range(len(x)-1):
    a, b, c, d = cs.c[:, i]
    formula = a*(X-x[i])**3 + b*(X-x[i])**2 + c*(X-x[i]) + d
    formula = sp.expand(formula)

    print(f"{x[i]} <= x <= {x[i+1]}")
    print(sp.N(formula, 10))
    print()

```

شکل 4

سوال 2)

ساختار و روند پیاده‌سازی کد پایتون

کد پروژه با استفاده از کتابخانه‌های استاندارد علم داده در پایتون شامل pandas (جهت مدیریت ساختار داده‌ها و لود اکسل)، numpy (محاسبات عددی)، matplotlib و seaborn (جهت رسم نمودارهای پیشرفته) توسعه یافته است. مراحل پیاده‌سازی به شرح زیر است:

گام اول: خوانش پویای فایل اکسل

کد ابتدا فایل اکسل با نام p3-s2.xlsx را فراخوانی می‌کند. برای جلوگیری از خطاهای احتمالی ناشی از وجود فاصله‌های خالی (Spaces) در نام ستون‌های فایل اصلی، از متد str.strip() استفاده شده است تا نام ستون‌ها به صورت استاندارد یکدست شوند.

گام دوم: محاسبات آماری (بخش ج)

زمان‌های اجرای مربوط به الگوریتم ۲ (Alg.2) به ازای ورودی‌های ۱۰۰ تا ۶۰۰ کیلوبایت مستقیماً از دیتافریم استخراج شده و با متد mean() میانگین‌گیری می‌شوند.

گام سوم: رسم نمودارهای اولیه (بخش الف)

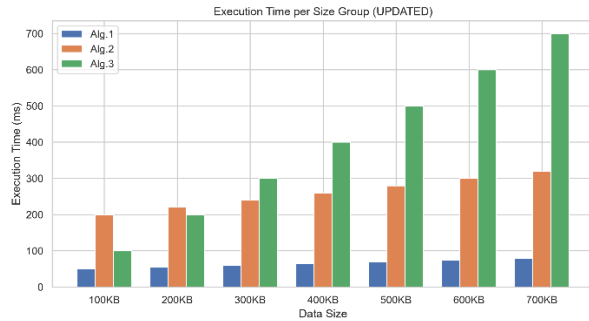
تابع generate_and_save_plots به صورت پویا طراحی شده تا با دریافت دیتافریم، سه نمودار خطی، میله‌ای و جعبه‌ای را رسم و با کیفیت بالا (300 DPI) در پوشه پروژه ذخیره کند. در نمودار جعبه‌ای، هشدارهای مربوط به به‌روزرسانی‌های آینده کتابخانه Seaborn با تنظیم پارامترهای hue و legend کاملاً برطرف شده است.

گام چهارم: تزریق داده‌های جدید و به‌روزرسانی (بخش ب)

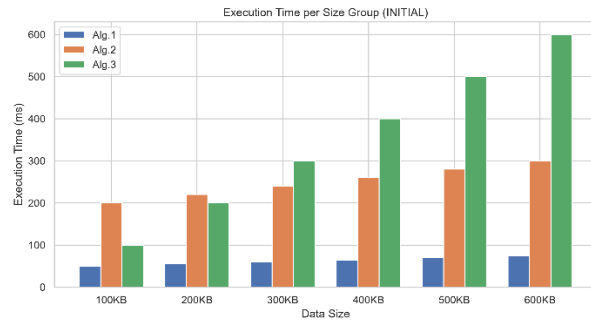
بدون تغییر در منطق کد، ردیف جدید مربوط به داده‌های KB700 در قالب یک دیتافریم مجزا تعریف شده و با استفاده از متد pd.concat به داده‌های قبلی متصل می‌گردد. سپس تابع رسم نمودار مجدداً فراخوانی می‌شود تا نمودارهای جدید حاوی داده‌های ۷۰۰ کیلوبایت به طور خودکار آپدیت و ذخیره شوند.

خروجی ترمینال پس از اجرای کد:

Average execution time for Alg.2 (100KB-600KB): 250.0



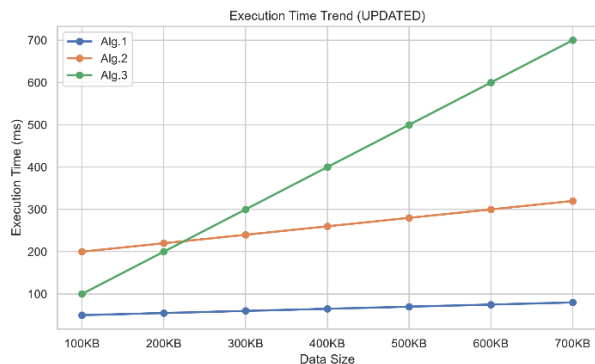
(ب) جدول با دیتای افزوده



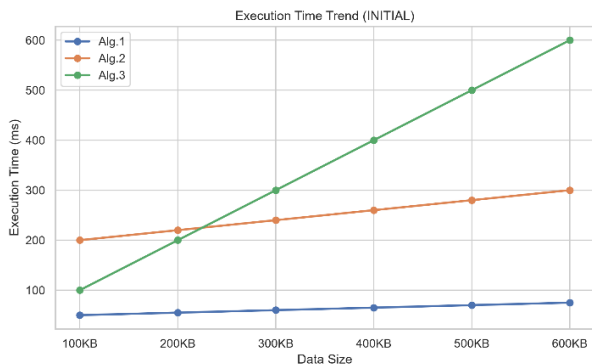
(الف) جدول با دیتای اولیه

شکل 5

تحلیل نمودار میله‌ای: این نمودار شکل 5 مقایسه رو در رو و گام به گام الگوریتم‌ها را در هر حجم ورودی مشخص نشان می‌دهد. همان‌طور که مشاهده می‌شود، در ورودی‌های کوچک (۱۰۰ کیلوبایت) اختلاف زمان اجرا ناچیز است، اما هرچه حجم داده‌ها به سمت ۶۰۰ و ۷۰۰ کیلوبایت حرکت می‌کند، ارتفاع میله مربوط به Alg.3 به شدت صعود کرده و تفاوت فاحشی با Alg.1 ایجاد می‌کند. این موضوع نشان می‌دهد الگوریتم ۱ کمترین میزان مصرف منابع را در مقیاس‌های بزرگتر دارد.



(ب) جدول با دیتای افزوده

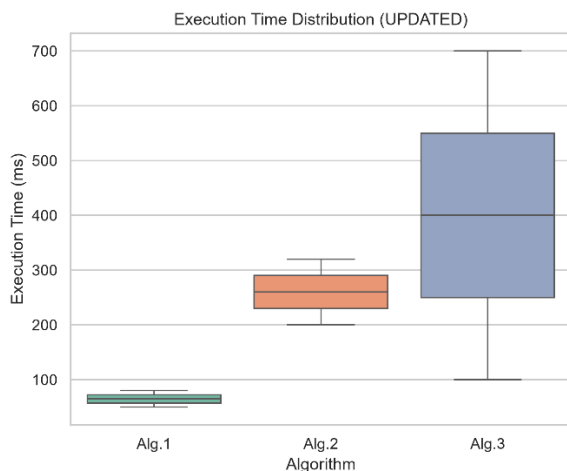


(الف) جدول با دیتای اولیه

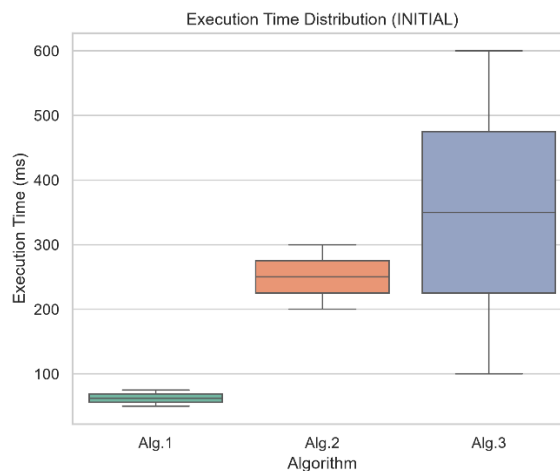
شکل 6

تحلیل نمودار خطی شکل 6: این نمودار رفتار رفتاری و پایداری الگوریتم‌ها را به تصویر می‌کشد.

- **الگوریتم ۱: (Alg.1)** خطی تقریباً افقی و با شیب بسیار ملایم (صعود ۵ واحدی در هر گام) دارد که نشان‌دهنده پیچیدگی بسیار بهینه (مانند $O(\log n)$ یا خطی ضعیف) است. با اضافه شدن ورودی ۷۰۰ کیلوبایت نیز این ثبات کاملاً حفظ شده است.
- **الگوریتم ۲: (Alg.2)** رفتاری کاملاً خطی ($O(n)$) با شیب ثابت ۲۰ واحد صعود در هر ۱۰۰ کیلوبایت دارد. اضافه شدن داده جدید در نقطه ۷۰۰ کیلوبایت (با زمان ۳۲۰) دقیقاً روی امتداد همان خط قبلی قرار گرفته و صحت الگوی رفتار آن را تایید می‌کند.
- **الگوریتم ۳: (Alg.3)** دارای پیچیدگی خطی با شیب شدید (۱۰۰ واحد صعود به ازای هر ۱۰۰ کیلوبایت) است. نمودار خطی به وضوح نشان می‌دهد که سرعت افت کارایی این الگوریتم با افزایش حجم داده، بسیار سریع‌تر از دو الگوریتم دیگر است و در ورودی ۷۰۰ کیلوبایت به اوج خود (۷۰۰ واحد زمان) رسیده است.



ب) جدول با دیتای افزوده



الف) جدول با دیتای اولیه

شکل 7

تحلیل نمودار جعبه‌ای: نمودار جعبه‌ای توزیع آماری و میزان پراکندگی داده‌های زمان اجرا را بدون در نظر گرفتن فاکتور ترتیب ابعاد نشان می‌دهد.

- جعبه مربوط به Alg.1 بسیار فشرده و در پایین‌ترین سطح نمودار قرار دارد که گویای کمترین نوسان، بالاترین پایداری و کارایی بی‌رقیب آن است.
- جعبه مربوط به Alg.3 بیشترین طول (پراکندگی یا Variance بالا) را به خود اختصاص داده است که نشان می‌دهد زمان اجرای آن به شدت به تغییرات ابعاد ورودی وابسته است.
- با اضافه شدن داده جدید در نسخه آپدیت شده (updated)، بدنه و خط میانه (Median) هر سه جعبه به دلیل افزایش مقادیر به سمت بالا متمایل شده‌اند، اما هیچ نقطه مجزایی به عنوان داده پرت یا Outlier شناسایی نشده است که نشان می‌دهد داده ۷۰۰ کیلوبایت با رفتار سیستماتیک الگوریتم‌ها همخوانی دارد.

سوال 3

هدف این گزارش، محاسبه انتگرال تابع $f(x) = e^{x^2}$ در بازه $(0,1)$ با استفاده از سه روش عددی «دوزنقه‌ای»، «سیمپسون» و «انتگرال‌گیری گاوسی» در پایتون و مقایسه دقت آن‌ها است.

نحوه پیاده‌سازی

برای پیاده‌سازی این سه روش، از متدهای آماده و بهینه‌سازی‌شده در کتابخانه علمی `scipy.integrate` استفاده شده است:

- **تعریف تابع و بازه:** ابتدا تابع $f(x) = e^{x^2}$ به صورت یک تابع خطی (`lambda`) و بازه $[0,1]$ به همراه ۱۰۰ زیربازه ($n = 100$) تعریف شد.
- **تولید نقاط (مش‌بندی):** با استفاده از `np.linspace`، بازه به نقاط هم‌فاصله تقسیم شد تا مقادیر تابع (y) در این نقاط محاسبه شود.
- **روش دوزنقه‌ای (Trapezoid):** با پاس دادن آرایه‌های x و y به متد آماده `trapezoid`، مساحت زیر نمودار با فرض خطی بودن فواصل تقریب زده شد.
- **روش سیمپسون (Simpson):** با پاس دادن همان آرایه‌ها به متد آماده `simpson`، تقریب با استفاده از منحنی‌های درجه ۲ (سه‌می) انجام شد.
- **انتگرال‌گیری گاوسی (Gaussian):** با پاس دادن خود تابع به متد هوشمند `quad`، انتگرال با بالاترین دقت عددی استاندارد محاسبه شد.

```
Trapezoid Method: 1.46269705
Simpson Method:   1.46265175
Gaussian Method:  1.46265175
Trapezoid Error:  4.53e-05
Simpson Error:    3.02e-09
```

شکل 8

تحلیل و نتیجه‌گیری

- **روش دوزنقه‌ای:** کمترین دقت را دارد؛ زیرا منحنی تابع را با خطوط صاف تقریب می‌زند که برای تابع صعودی و رو به بالایی مثل e^{x^2} خطای بیشتری تولید می‌کند.
- **روش سیمپسون:** بسیار دقیق‌تر از دوزنقه‌ای عمل کرده و خطای آن چند مرتبه بزرگ‌تر کم کاهش یافته است؛ چرا که از منحنی‌های درجه ۲ برای فیت شدن روی نمودار استفاده می‌کند.
- **روش گاوسی:** به عنوان دقیق‌ترین روش و مرجع سنجش خطا در نظر گرفته شده و بدون نیاز به مش‌بندی دستی، به جواب دقیق کلیدی می‌رسد.